

Computer Vision and Deep Learning

Week 2 Manual

Image Filters and Introduction to Convolutional Concepts

Learning Objectives

By the end of this week, you will:

- Understand how image filters work and their applications
- Explore different types of filters for edge detection, blurring, and enhancement
- Learn the basic concept of convolution in image processing
- Understand why spatial relationships matter in images
- Build simple pattern recognition using traditional methods

Prerequisites:

- Completion of Week 1 assignments
- Basic understanding of image processing and neural networks
- Familiarity with Python, NumPy, and OpenCV

June 30, 2025

Contents

1	Assignment 1: Image Filters and Feature Detection	2
1.1	Learning Material	2
1.1.1	What Are Image Filters?	2
1.1.2	How Filters Work	2
1.1.3	Common Filter Types	2
1.2	Demo Module	2
1.2.1	Exploring Different Filters	2
1.3	Task Modules	3
1.3.1	Task 1.1: Filter Exploration (Monday)	3
1.3.2	Task 1.2: Simple Pattern Detection (Tuesday)	4
1.4	Assessment Criteria for Assignment 1	4
2	Assignment 2: Understanding Convolution and Spatial Relationships	5
2.1	Learning Material	5
2.1.1	What is Convolution?	5
2.1.2	Why Spatial Relationships Matter	5
2.1.3	Building Towards Neural Networks	5
2.2	Demo Module	5
2.2.1	Simple Convolution Implementation	5
2.3	Task Modules	7
2.3.1	Task 2.1: Manual Convolution Practice (Wednesday)	7
2.3.2	Task 2.2: Kernel Experiment (Thursday)	7
2.4	Assessment Criteria for Assignment 2	8
3	Weekly Assessment (Friday)	8
4	Recommended Readings	8
5	Next Week Preview	8

1 Assignment 1: Image Filters and Feature Detection

Duration: Monday - Tuesday — **Assessment:** Tuesday End of Day

1.1 Learning Material

1.1.1 What Are Image Filters?

Image filters are mathematical operations that modify pixel values based on their neighbors. They're the foundation of many computer vision techniques:

- **Smoothing filters:** Reduce noise and blur images
- **Edge detection filters:** Highlight boundaries and transitions
- **Sharpening filters:** Enhance image details
- **Custom filters:** Detect specific patterns or features

1.1.2 How Filters Work

A filter is a small matrix (usually 3×3 or 5×5) that slides across an image:

1. Place filter over a region of the image
2. Multiply corresponding pixels with filter values
3. Sum all products to get new pixel value
4. Move filter to next position and repeat

1.1.3 Common Filter Types

Edge Detection:

- Sobel filters: Detect horizontal and vertical edges
- Laplacian filter: Detect edges in all directions
- Canny edge detector: Advanced multi-step edge detection

Smoothing:

- Box filter: Simple averaging
- Gaussian filter: Natural-looking blur
- Median filter: Removes noise while preserving edges

1.2 Demo Module

1.2.1 Exploring Different Filters

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Load and prepare image
6 image = cv2.imread('sample_image.jpg')
7 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
8 gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
9
10 # Define different filters
11 filters = {
12     'Original': gray,
13     'Gaussian Blur': cv2.GaussianBlur(gray, (15, 15), 0),
14     'Horizontal Edges': cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3),
15     'Vertical Edges': cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3),
```

```

16     'All Edges': cv2.Laplacian(gray, cv2.CV_64F),
17     'Canny Edges': cv2.Canny(gray, 50, 150)
18 }
19
20 # Create visualization
21 fig, axes = plt.subplots(2, 3, figsize=(15, 10))
22 axes = axes.flatten()
23
24 for i, (name, filtered_img) in enumerate(filters.items()):
25     axes[i].imshow(filtered_img, cmap='gray')
26     axes[i].set_title(name, fontsize=12)
27     axes[i].axis('off')
28
29 plt.tight_layout()
30 plt.show()
31
32 # Demonstrate filter effects on different image types
33 def test_filter_on_shapes():
34     """Test filters on simple geometric shapes"""
35     # Create a simple image with shapes
36     img = np.zeros((200, 200), dtype=np.uint8)
37
38     # Draw shapes
39     cv2.rectangle(img, (50, 50), (100, 100), 255, -1) # Square
40     cv2.circle(img, (150, 150), 30, 128, -1) # Circle
41
42     # Apply different filters
43     blurred = cv2.GaussianBlur(img, (15, 15), 0)
44     edges = cv2.Canny(img, 50, 150)
45
46     # Display results
47     fig, axes = plt.subplots(1, 3, figsize=(12, 4))
48     axes[0].imshow(img, cmap='gray')
49     axes[0].set_title('Original Shapes')
50     axes[1].imshow(blurred, cmap='gray')
51     axes[1].set_title('Blurred')
52     axes[2].imshow(edges, cmap='gray')
53     axes[2].set_title('Edge Detected')
54
55     for ax in axes:
56         ax.axis('off')
57     plt.tight_layout()
58     plt.show()
59
60 test_filter_on_shapes()

```

Listing 1: Image Filters with OpenCV

Expected Output: Visual comparison showing how different filters affect the same image, plus demonstration on simple geometric shapes.

1.3 Task Modules

1.3.1 Task 1.1: Filter Exploration (Monday)

1. Choose one image and apply these three filters:
 - Gaussian blur (kernel size 15)
 - Canny edge detection
 - Laplacian edge detection
2. Display the original and three filtered versions in a 2×2 grid
3. Write 2-3 sentences about what each filter does to your image

Deliverable: Python script showing 4 images (original + 3 filters) with brief descriptions

1.3.2 Task 1.2: Simple Pattern Detection (Tuesday)

1. Create a simple black and white image with basic shapes (circles, squares, triangles, etc. using drawing tools like MS Paint or draw.io)
2. Apply Canny edge detection to find the edges
3. Compare the result with the original shapes

Deliverable: Before/after images showing edge detection on your drawn shapes

1.4 Assessment Criteria for Assignment 1

- **Correct Implementation (50%):** Filters applied correctly with proper display
- **Understanding (30%):** Clear descriptions of what each filter does
- **Presentation (20%):** Clean, well-organized output with good image choices

2 Assignment 2: Understanding Convolution and Spatial Relationships

Duration: Wednesday - Thursday — **Assessment:** Thursday End of Day

2.1 Learning Material

2.1.1 What is Convolution?

Convolution is the mathematical operation behind image filtering. It's important because:

- It preserves spatial relationships between pixels
- It can detect features regardless of their position in the image
- It's the foundation of Convolutional Neural Networks (CNNs)
- It allows sharing of learned features across the entire image

2.1.2 Why Spatial Relationships Matter

Unlike traditional neural networks that treat images as flat lists of pixels, convolution respects the 2D structure:

- Nearby pixels are related to each other
- Patterns can appear anywhere in the image
- Local features combine to form complex objects
- Translation invariance: same feature detected anywhere

2.1.3 Building Towards Neural Networks

Traditional approach: Hand-designed filters → Neural network approach: Learned filters

- Instead of manually designing edge detectors, let the network learn them
- Multiple filters can be learned simultaneously
- Filters can be combined in layers to detect complex patterns
- The network learns the best filters for the specific task

2.2 Demo Module

2.2.1 Simple Convolution Implementation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import cv2
4
5 def simple_convolution(image, kernel):
6     """
7     Simple convolution implementation for learning purposes
8     """
9     # Get image and kernel dimensions
10    img_height, img_width = image.shape
11    kernel_height, kernel_width = kernel.shape
12
13    # Calculate output size (without padding)
14    output_height = img_height - kernel_height + 1
15    output_width = img_width - kernel_width + 1
16
17    # Initialize output
18    output = np.zeros((output_height, output_width))
```

```

19
20     # Perform convolution
21     for i in range(output_height):
22         for j in range(output_width):
23             # Extract the region
24             region = image[i:i+kernel_height, j:j+kernel_width]
25             # Element-wise multiplication and sum
26             output[i, j] = np.sum(region * kernel)
27
28     return output
29
30 # Create a simple test image
31 test_image = np.array([
32     [1, 1, 1, 0, 0],
33     [0, 1, 1, 1, 0],
34     [0, 0, 1, 1, 1],
35     [0, 0, 1, 1, 0],
36     [0, 1, 1, 0, 0]
37 ], dtype=np.float32)
38
39 # Simple edge detection kernel
40 edge_kernel = np.array([
41     [-1, -1, -1],
42     [-1, 8, -1],
43     [-1, -1, -1]
44 ], dtype=np.float32)
45
46 # Apply convolution
47 result = simple_convolution(test_image, edge_kernel)
48
49 # Visualize
50 fig, axes = plt.subplots(1, 3, figsize=(12, 4))
51
52 axes[0].imshow(test_image, cmap='gray')
53 axes[0].set_title('Original Image')
54 axes[0].set_xlabel('5x5 pixels')
55
56 axes[1].imshow(edge_kernel, cmap='RdBu')
57 axes[1].set_title('Edge Detection Kernel')
58 axes[1].set_xlabel('3x3 kernel')
59
60 axes[2].imshow(result, cmap='gray')
61 axes[2].set_title('Convolution Result')
62 axes[2].set_xlabel('3x3 output')
63
64 for ax in axes:
65     ax.axis('off')
66
67 plt.tight_layout()
68 plt.show()
69
70 # Demonstrate on real image
71 def demo_multiple_kernels():
72     """Show how different kernels detect different features"""
73
74     # Load image
75     img = cv2.imread('sample_image.jpg', cv2.IMREAD_GRAYSCALE)
76     img = cv2.resize(img, (150, 150)) # Keep it small for clarity
77
78     # Define different kernels
79     kernels = {
80         'Vertical Lines': np.array([[ -1, 2, -1],
81                                     [ -1, 2, -1],

```

```

82         [-1, 2, -1]],
83
84         'Horizontal Lines': np.array([[ -1, -1, -1],
85                                       [ 2, 2, 2],
86                                       [-1, -1, -1]]),
87
88         'Diagonal': np.array([[ -1, -1, 2],
89                               [-1, 2, -1],
90                               [ 2, -1, -1]]),
91
92         'Sharpen': np.array([[ 0, -1, 0],
93                              [-1, 5, -1],
94                              [ 0, -1, 0]])
95     }
96
97     fig, axes = plt.subplots(2, 3, figsize=(15, 10))
98     axes = axes.flatten()
99
100    # Show original
101    axes[0].imshow(img, cmap='gray')
102    axes[0].set_title('Original Image')
103    axes[0].axis('off')
104
105    # Apply each kernel
106    for i, (name, kernel) in enumerate(kernels.items(), 1):
107        result = cv2.filter2D(img, -1, kernel)
108        axes[i].imshow(result, cmap='gray')
109        axes[i].set_title(f'{name} Detection')
110        axes[i].axis('off')
111
112    axes[5].axis('off') # Hide the last subplot
113    plt.tight_layout()
114    plt.show()
115
116    demo_multiple_kernels()

```

Listing 2: Understanding Convolution Step by Step

Expected Output: Step-by-step visualization of convolution process and demonstration of how different kernels detect different features.

2.3 Task Modules

2.3.1 Task 2.1: Manual Convolution Practice (Wednesday)

1. Use the provided simple convolution function from the demo
2. Create one small test image (5×5) with a simple pattern
3. Apply the edge detection kernel to your test image
4. Compare your manual result with OpenCV's `cv2.filter2D()` on the same image

Deliverable: Simple test showing manual convolution vs OpenCV result

2.3.2 Task 2.2: Kernel Experiment (Thursday)

1. Take one real image and apply these three different kernels:

- Vertical line detector: $\begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$
- Horizontal line detector: $\begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$

- Sharpen filter: $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$

2. Display the results and observe which features each kernel highlights

Deliverable: One image with three different kernel effects applied

2.4 Assessment Criteria for Assignment 2

- **Manual Convolution (40%):** Correct implementation and comparison with OpenCV
- **Kernel Application (35%):** Proper use of different kernels on real images
- **Understanding (25%):** Basic grasp of how different kernels detect different features

3 Weekly Assessment (Friday)

Comprehensive evaluation covering both assignments

The weekly assessment will focus on understanding filter applications and the convolution concept, preparing students for more advanced topics in coming weeks.

4 Recommended Readings

- "Computer Vision: Algorithms and Applications" by Richard Szeliski (Chapter 3: Image Processing)
- OpenCV documentation on filtering functions
- Online tutorials on convolution visualization
- "Digital Image Processing" by Gonzalez & Woods (basic chapters)

5 Next Week Preview

Week 3 will cover:

- Introduction to modern deep learning frameworks
- Building your first Convolutional Neural Network
- Training networks to learn filters automatically
- Simple image classification projects

**Focus on understanding how filters work and why spatial relationships matter.
These concepts are the foundation for everything that follows!**